

Talk Structurally, Act Hierarchically: A Collaborative Refinement Framework for LLM Multi-Agent Systems

Zhao Wang¹, Sota Moriyama², Wei-Yao Wang¹, Briti Gangopadhyay¹, and Shingo Takamatsu¹

Abstract—While recent advancements in large language model-based multi-agent (LLM-MA) systems have shown promise, significant challenges remain in managing communication and refinement when agents collaborate on complex tasks. In this article, we propose *talk structurally, act hierarchically (TalkHier)*, a novel framework that introduces a structured communication protocol for context-rich exchanges and a hierarchical refinement system to address issues such as incorrect outputs, falsehoods, and biases. *TalkHier* surpasses various types of SoTA, including inference scaling model (OpenAI-o1), open-source multiagent models (e.g., AgentVerse), and majority voting strategies on current LLM and single-agent baselines (e.g., ReAct and GPT4o), across diverse tasks, including open-domain question answering, domain-specific selective questioning, and practical advertisement text generation. These results highlight its potential to set a new standard for LLM-MA systems, paving the way for more effective, adaptable, and collaborative multiagent frameworks.

Impact Statement—LLM-MA systems are increasingly used in collaborative reasoning and decision-making. However, current approaches struggle with unstructured communication and inflexible refinement schemes, which limit their effectiveness on complex, multistep tasks. This article introduces *TalkHier*, a novel framework that addresses these limitations through a structured communication protocol and a dynamic hierarchical refinement system. By organizing agent interactions around clear message structures and role-specific evaluation workflows, *TalkHier* enhances coordination and robustness in multiagent settings. Empirical results demonstrate that the proposed method outperforms existing single-agent and multiagent baselines on diverse benchmarks such as open-domain QA, reasoning-heavy tasks, and Japanese ad text generation. The framework's structured communication and hierarchical refinement make it more robust and generalizable than existing multiagent systems, enabling reliable performance across diverse domains.

Index Terms—Hierarchical refinement, large language models (LLMs), multiagent systems, structured communication protocol.

Received 3 January 2026; revised 13 February 2026; accepted 16 March 2026. Date of publication 20 March 2026; date of current version 31 August 2026. This article was recommended for publication by Associate Editor Dongbin Zhao upon evaluation of the reviewers' comments. (Zhao Wang and Sota Moriyama contributed equally to this work.) (Sota Moriyama completed this work during his internship at Sony.) (Corresponding author: Zhao Wang.)

Zhao Wang, Wei-Yao Wang, Briti Gangopadhyay, and Shingo Takamatsu are with Sony Group Corporation, Shinagawa City, Tokyo 141-8610, Japan (e-mail: Zhao.Wang@sony.com).

Sota Moriyama is with the Graduate University for Advanced Studies, Shonankokusaimura, Hayama, Miura District, Kanagawa 240-0115, Japan. He was with Sony Group Corporation, Shinagawa City, Tokyo 141-8610, Japan. Digital Object Identifier 10.1109/TAI.2026.3676025

I. INTRODUCTION

LARGE language model (LLM) agents have broad applications across domains such as robotics [1], finance [2], [3], and coding [4], [5]. By enhancing capabilities such as autonomous reasoning [6] and decision-making [7], LLM agents bridge the gap between human intent and machine execution, generating contextually relevant responses [8].

Recent research has primarily focused on LLM-based multi-agent (LLM-MA) systems, which leverage collective intelligence and specialize each agent with the corresponding subtasks, to solve complicated and multistep problems. For instance, previous works on LLM-MA have explored approaches where instances of LLMs, referred to as agents [9], [10], [11], [12], [13], collaborate synergistically by debate [14], reflection [15], self-refinement [16], or multiagent-based feedback refinement [17]. These systems employ diverse communication topologies to enable efficient interactions between agents such as Chain [18] and Tree [19] structures, among others [20], [21], [22].

Despite the promising advancements in LLM-MA systems, several challenges in this field remain unexplored (shown in Fig. 1), as follows.

A. Disorganized Communication in Text Form

Agents often engage in debates [23], share insights [14], or perform refinement [16], [17] to effectively solve complex tasks, with their exchanges primarily in text form [24]. However, communication often becomes disorganized because it requires explicitly describing agent tasks, providing background context for the communication, and specifying the required output formats. These factors together lead to lengthy and unstructured exchanges, making it difficult for agents to manage subgoals, maintain output structures, and retrieve independent memories from prior actions and observations.

B. Refinement Schemes

While some studies have shown that incorporating agent debates [14] or evaluation-based multiagent refinement [17], [25] can improve system accuracy, these approaches also exhibit significant limitations. As the number of agents increases, LLM-MA systems face challenges in effectively summarizing opinions or feedback [26]. Static hierarchical refinement

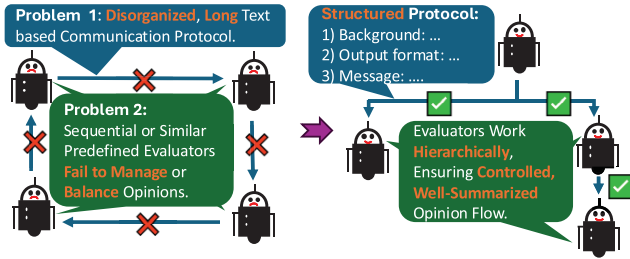


Fig. 1. Existing LLM-MA methods (left) face two major challenges: 1) disorganized, lengthy text-based communication protocols; and 2) sequential or static hierarchical multiagent refinements. In contrast, *TalkHier* (right) introduces a well-structured communication protocol and a dynamic hierarchical refinement approach.

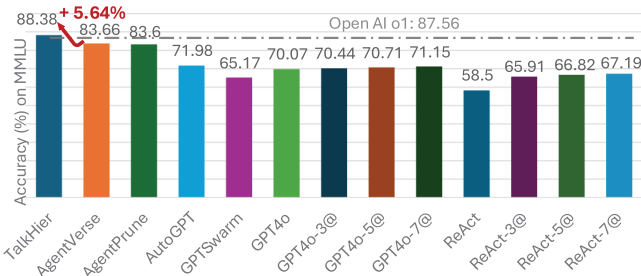


Fig. 2. Our *TalkHier* built on GPT4o surpasses inference scaling models (OpenAI-o1), open-source multiagent models (AgentVerse, etc.), and models with majority voting strategies (ReAct, GPT4o) on five subtasks of MMLU.

methods, such as AgentVerse [27], rely on predefined fixed hierarchies (requiring a rigid four-stage refinement process that must be followed sequentially: expert recruitment, collaborative decision-making, action execution, and evaluation) and often lead to suboptimal results, as agents may fail to properly balance diverse inputs—frequently overlooking some or exhibiting biases based on the order in which feedback is presented [28].

In this article, we propose a novel collaborative LLM-MA framework called *talk structurally, act hierarchically* (*TalkHier*)-the first collaborative LLM-MA framework to integrate a well-structured communication protocol with hierarchical refinement. Our key contributions shown in Figs. 1 and 2 are as follows.

- 1) *Well-Structured, Context-Rich Communication Protocol*: *TalkHier* introduces a novel communication protocol that incorporates newly proposed elements: *messages*, *intermediate outputs*, and relevant *background information*. These components form the foundation of a well-structured protocol that organizes agent communication, ensuring clarity and precision. By embedding these elements, *TalkHier* significantly improves communication accuracy and efficiency compared to traditional text-based methods.
- 2) *Hierarchical Refinement with Dynamic Routing*: *TalkHier* enhances traditional multiagent evaluation systems through a hierarchical refinement framework that enables agents to act dynamically without user-defined configurations. This approach addresses challenges such as the difficulty of summarizing opinions and feedback

as the number of agents increases, balancing diverse inputs, and mitigating biases caused by the order in which feedback is processed, resulting in more reliable and robust interactions.

- 3) *State-of-the-Art Results Across Benchmarks*: Experimental results show that *TalkHier* achieves state-of-the-art performance on diverse benchmarks, including selective problem-solving in complex subdomains, open question answering, and Japanese text generation tasks. Ablation studies confirm the effectiveness of each component, demonstrating their contributions to the framework's overall success.

II. RELATED WORK

A. LLM-MA Systems

LLM-MA systems enable agents to collaborate on complex tasks through dynamic role allocation, inter-agent connections (defining the system topology), communication, and task execution [24], [29]. Recent advancements include agent profiling [30], hierarchical communication [31], and integration of reasoning and intentions [32]. However, challenges remain in ensuring robust communication, avoiding redundancy, and refining evaluation processes [33]. Standardized benchmarks and frameworks are needed to drive future progress [34].

1) *Connections and Topology in LLM-MA*: The structure of a multiagent system is defined by the connections (edges) between agents, which determine the possible pathways for information flow. Many previous works have focused on the design of these topologies, such as chain [18], tree [19], complete graph [20], random graph [20], optimizable graph [21], and pruned graph [22] structures. The choice of topology affects how agents can interact, coordinate, and share information.

Some recent approaches, such as AgentPrune [22] and GPTSwarm [21], attempt to find optimal or near-optimal connection structures through training. These methods use learning-based strategies to adapt the agent connectivity for improved performance on specific tasks. However, a key computational limitation is that the connection structure must be re-trained or re-optimized for each new task or domain, which can be resource-intensive and may not scale efficiently to diverse or rapidly changing problem settings.

2) *Communication Protocols in LLM-MA*: While the connections define who can communicate with whom, the communication protocol specifies how information is actually exchanged over these connections. Most existing systems rely on text-based exchanges [22], [35], which can be inefficient and error-prone as agents may lose track of subtasks or fail to recall prior outputs as tasks grow in complexity. There has been limited discussion on the optimal form of communication. We argue for structured communication protocols that guide subtasks with clear, context-specific instructions, ensuring coherence and efficiency across interactions.

B. Feedback-Based Refinement

Feedback mechanisms, such as self-refine [16], generator-evaluator frameworks [25], and predefined agent

role-based refinement frameworks [14], [22] improve system accuracy through iterative refinement. However, these methods face challenges in managing diverse feedback, which can lead to bias or inefficiencies [36]. They are sensitive to the order of evaluation, often placing greater emphasis on the most recent evaluation result when multiple agents are involved [34], [37].

Other structured refinement approaches, such as AgentVerse [27], switch between horizontal and vertical structures depending on the task. However, its static predefined hierarchical design (a rigid four-stage refinement process that must be followed sequentially: expert recruitment, collaborative decision-making, action execution, and evaluation) restricts flexibility and limits its ability to support more complex evaluation pipelines involving dynamic routing. This often leads to overlooked feedback or biases in the refinement process [28].

III. METHODOLOGY

In this section, we detail the design and operational principles of the proposed *TalkHier* framework, including its agent structure, hierarchical collaboration, and structured communication protocol. As shown in Fig. 3, after the user inputs a task description along with one phrase-based metrics and their corresponding one sentence-based descriptions, *TalkHier* aims to generate an answer by automatically constructing an LLM-MA system, represented as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes the set of agents (nodes), and $\mathcal{E}$ represents the set of communication pathways (edges).

Unbiasedness Across Stages: In this work, by *unbiasedness* we refer to mitigating two protocol-induced biases that commonly arise in LLM-MA refinement: 1) *order-/position-induced bias* in long-context communication, where earlier or more salient feedback is over-weighted; and 2) *anchoring/confirmation effects*, where early drafts or strong opinions become anchors that subsequent agents tend to rationalize. Unlike other refinement systems such as the single-agent self-refine [16], role-defined flat multiagent refinement [14], [22], [25], or static hierarchical multiagent refinement [27], *TalkHier* adopts a more flexible structure: a basic generator $v_{\text{Gen}}^{\text{main}}$, an evaluation team $\mathcal{V}^{\text{eval}}$ (which may consist of multiple evaluation agents), and a revisor $v_{\text{Rev}}^{\text{main}}$. This topology reflects the natural structure of many problems, which can be viewed as a process of generation, evaluation, and revision. One of the core novelties of *TalkHier* lies in enabling these three stages to operate with reduced protocol-induced bias, specifically, by: 1) performing *parallel, metric-scoped* evaluations to avoid a single sequential debate stream; 2) aggregating feedback through supervisor-level verification signals rather than a single long, order-sensitive context; and 3) ensuring that diverse evaluator inputs are preserved instead of being overlooked due to their presentation order. Details on how *TalkHier* achieves the hierarchical refinement will be presented in Section III-B.

Another key novelty lies in the communication protocol between agents. Given an input problem p , the system dynamically defines a set of communication events $\mathcal{C}_p$, where each event $c_{ij}^{(t)} \in \mathcal{C}_p$ represents a communication between agents v_i and v_j along an edge $e_{ij} \in \mathcal{E}$ at time step t . As mentioned

earlier, unlike traditional LLM agent systems [14], [22], [27], [38], which rely on lengthy, unstructured text-based communication, *TalkHier* introduces a well-organized and structured communication protocol. Details on the design of each $c_{ij}^{(t)}$ will be presented in Section III-C.

A. Agent Notation

Each agent $v_i \in \mathcal{V}$ can be formally represented as

$$v_i = (\text{Role}_i, \text{Plugins}_i, \text{Memory}_i, \text{Type}_i)$$

where the components are defined as follows.

- 1) Role_i : Assign roles such as generator, evaluator, or revisor based on the task type.
- 2) Plugins_i : External tools or plugins attached for domain-specific operations.
- 3) Memory_i : An agent-specific memory that stores and retrieves information relevant to the agent's role and task.
- 4) Type_i : Specifies whether the agent is a supervisor (S) responsible for overseeing task success, or a member (M) focused on problem-solving.

The first three components— Role_i , Plugins_i , and Memory_i —are standard in most related works, forming the foundation of agent functionality. Our contributions lie in the last component: Type_i , which explicitly categorizes agents into supervisor (S) roles, responsible for overseeing the multiagent team and ensuring task success, or member (M) roles, focused on problem-solving and optionally utilizing plugins. These additions enable hierarchical, structured collaboration and role-specific operations within the framework.

Agent-Specific Memory: While some prior works such as LangGraph [39] and GPTSwarm [21] utilize a shared memory pool accessible by all agents, we follow the approach in [40] and adopt agent-specific memory. In this design, each agent maintains a private memory buffer that stores its own experiences, which are neither shared with nor accessible by other agents. This design choice in *TalkHier* is motivated by two key advantages: independence, where each agent's memory functions without interference or reliance on others, thus avoiding centralized bottlenecks; and persistence, which allows agents to maintain historical context across all conversations for consistent and informed decision-making.

B. Collaborative, Hierarchical Agent Team

The entire graph $\mathcal{G}$ consists of multiple teams, each represented as a subset $\mathcal{V}^{\text{team}} \subseteq \mathcal{V}$. Each team includes a dedicated supervisor agent v_S^{team} and one or more member agents v_M^{team} . A key feature of the hierarchical structure in *TalkHier* is that a member agent in one team can also act as a supervisor for another team, creating a nested hierarchy of agent teams. For example, as shown in Fig. 3, the evaluation team supervisor v_S^{eval} is a member agent in the team supervised by v_S^{main} , while also being a supervisor in another team leading two evaluators (factual accuracy evaluator v_{E1}^{eval} and policy alignment evaluator v_{E2}^{eval}). Formally, the hierarchical structure of agents with the described two teams is defined as

$$\mathcal{V}^{\text{main}} = \{v_S^{\text{main}}, v_{\text{Gen}}^{\text{main}}, v_S^{\text{eval}}, v_{\text{Rev}}^{\text{main}}\}$$

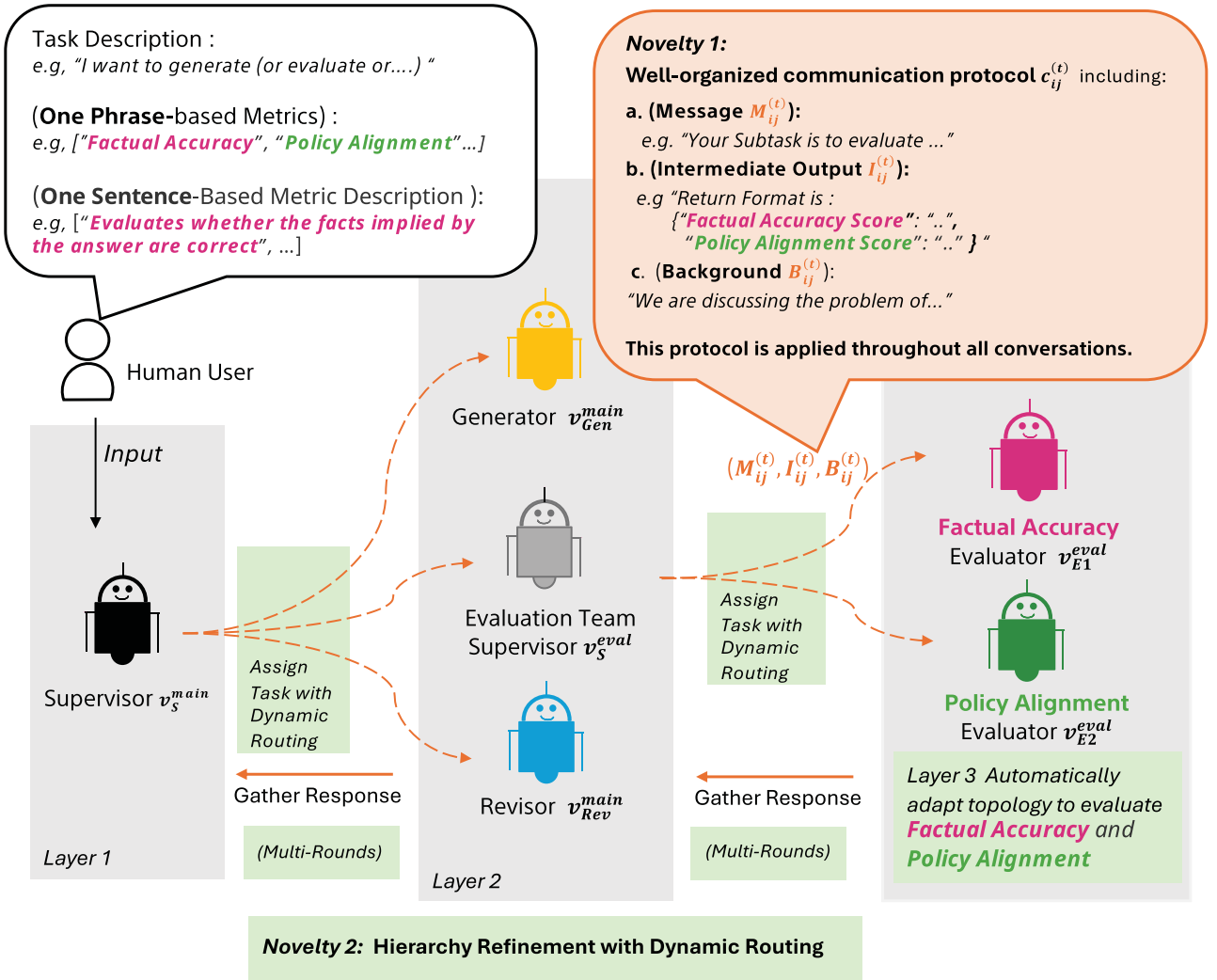


Fig. 3. Example illustration of *TalkHier* graph. It introduces a novel communication protocol (orange arrows and boxes), featuring well-structured communication $c_{ij}^{(t)}$ composed of our proposed elements ($M_{ij}^{(t)}$, $B_{ij}^{(t)}$, $I_{ij}^{(t)}$). The framework also incorporates a collaborative hierarchical refinement mechanism with dynamic routing (green box), implemented through a 3-layer agent team (gray box). This team includes a supervisor v_S^{main} in the first layer; an evaluation supervisor v_S^{eval} , a generator $v_{\text{Gen}}^{\text{main}}$, and a revisor $v_{\text{Rev}}^{\text{main}}$ in the second layer; and two independent evaluators v_{E1}^{eval} and v_{E2}^{eval} in the third layer, automatically instantiated based on the user-specified metrics. This dynamic hierarchical structure ensures unbiased and coordinated feedback throughout the framework.

$$\mathcal{V}^{\text{eval}} = \{v_S^{\text{eval}}, v_{E1}^{\text{eval}}, v_{E2}^{\text{eval}}, \dots, v_{E_k}^{\text{eval}}\}$$

where the main supervisor (v_S^{main}) and evaluation supervisor (v_S^{eval}) oversee their respective team's operations and assign tasks to each member, the generator ($v_{\text{Gen}}^{\text{main}}$) gives solutions for a given problem, and the revisor ($v_{\text{Rev}}^{\text{main}}$) refines outputs based on given feedback. Furthermore, the evaluation team consists of k independent evaluators, denoted as $v_{E_k}^{\text{eval}}$, where k is determined by the number of user-defined evaluation metrics. Each member of the evaluation team provides assessment results for a given problem based on their assigned metric.

1) How the Hierarchy Reduces Order and Anchoring Biases: The hierarchical design reduces protocol-induced biases by enforcing *parallel, metric-scoped* evaluation. Since evaluators $\{v_{E_j}^{\text{eval}}\}_j$ independently assess the same intermediate output under disjoint metrics, the framework is less prone to anchoring on a single early opinion than sequential

debate-style communication. Moreover, feedback is consolidated at the supervisor level via a summarized evaluation signal, which mitigates position effects that arise when many long feedback messages are concatenated into a single context window. This design also reduces the risk of overlooking specific evaluator inputs due to their presentation order.

2) Algorithm: Algorithm 1 details the hierarchical refinement process in our collaborative agent framework at iteration $t \in \{1, \dots, T\}$, where T is the maximum number of refinement iterations. The process starts with the main supervisor (v_S^{main}) assigning evaluation criteria to the evaluation supervisor (v_S^{eval}). Specifically, the assignment action at iteration t is denoted as T_{assign}^t , which specifies a set of evaluation metrics (or rubrics) $\{\text{Metric}_{E_j}\}_{j=1}^{|\mathcal{V}^{\text{eval}}|}$ distributed to the evaluator agents $\mathcal{V}^{\text{eval}} = \{v_{E_j}^{\text{eval}}\}$. Given the previous-stage system output $\mathbf{A}_{t-1}$ (with $\mathbf{A}_0$ being the initial draft), each evaluator $v_{E_j}^{\text{eval}}$

Algorithm 1: Hierarchical Refinement.

Input: Initial output $\mathbf{A}_0$ (from Generator agent $v_{\text{Gen}}^{\text{main}}$), quality threshold $\mathcal{M}_{\text{threshold}}$, maximum iterations T_{max}
Output: Final output $\mathbf{A}_{\text{final}}$

```

1 Set  $t \leftarrow 0$ 
2 Set  $\mathbf{A}_{\text{final}} \leftarrow \mathbf{A}_0$ 
3 while  $t < T_{\text{max}}$  do
4    $t \leftarrow t + 1$     $\mathbf{T}_{\text{assign}}^{(t)} = \{(\text{Roles}_S, \text{Criteria}_{v_S^{\text{eval}}})\}$  // Assign criteria from  $v_S^{\text{main}}$  to  $v_S^{\text{eval}}$ 
5   for each  $v_{E_j}^{\text{eval}} \in \mathcal{V}^{\text{eval}}$  do
6      $\mathbf{F}_{E_j}^{(t)} = f_{\text{evaluate}}(\mathbf{A}_{t-1}, \text{Metric}_{E_j})$  // Each evaluator provides feedback
7    $\mathbf{F}_{\text{eval}}^{(t)} = \{\mathbf{F}_{E_1}^{(t)}, \dots, \mathbf{F}_{E_k}^{(t)}\}$ 
8    $\mathbf{F}_{\text{summary}}^{\text{eval}} = f_{\text{summarize}}(\mathbf{F}_{\text{eval}}^{(t)})$  //  $v_S^{\text{eval}}$  aggregates and summarizes feedback
9   if  $\mathcal{M}(\mathbf{F}_{\text{summary}}^{\text{eval}}) \geq \mathcal{M}_{\text{threshold}}$  then // Check whether metric is above threshold
10     $\mathbf{A}_{\text{final}} = \mathbf{A}_{t-1}$  // Accept current output
11    break
12  else
13     $\mathbf{A}_t = f_{\text{revise}}(\mathbf{A}_{t-1}, \mathbf{F}_{\text{summary}}^{\text{eval}})$  // Revisor refines the output
14 return  $\mathbf{A}_{\text{final}}$ 

```

independently reviews $\mathbf{A}_{t-1}$ according to its assigned metric and produces feedback $\mathbf{F}_{E_j}^{(t)}$ via the evaluation function $f_{\text{evaluate}}(\cdot)$, i.e., $\mathbf{F}_{E_j}^{(t)} = f_{\text{evaluate}}(\mathbf{A}_{t-1}, \text{Metric}_{E_j})$. The evaluation supervisor then aggregates all evaluator feedback into a single summary $\mathbf{F}_{\text{summary}}^{\text{eval}}$ using the summarization function $f_{\text{summarize}}(\cdot)$, i.e., $\mathbf{F}_{\text{summary}}^{\text{eval}} = f_{\text{summarize}}(\{\mathbf{F}_{E_j}^{(t)}\}_j)$. We denote the resulting evaluation signal at iteration t as $\mathbf{F}_{\text{eval}}^t$, which is instantiated by $\mathbf{F}_{\text{summary}}^{\text{eval}}$ and sent to the main supervisor for decision making. The main supervisor checks whether the summarized feedback satisfies the predefined quality threshold $\mathcal{M}_{\text{threshold}}$; if satisfied, the current output $\mathbf{A}_{t-1}$ is accepted as final. Otherwise, the revisor ($v_{\text{Rev}}^{\text{main}}$) refines the output to produce the next output $\mathbf{A}_t$, and the process repeats. This loop continues until the output meets $\mathcal{M}_{\text{threshold}}$ or reaches T iterations, enabling systematic refinement while reducing order-induced and anchoring effects through multiview evaluation and aggregation.

The main supervisor determines whether the summarized feedback meets the predefined quality threshold ($\mathcal{M}_{\text{threshold}}$)—for example, by ensuring correctness or achieving high relevance. If the threshold is met, the current output is accepted as final. If not, the revisor ($v_{\text{Rev}}^{\text{main}}$) refines the output, and the process repeats until the threshold is satisfied or the maximum number of iterations is reached.

C. Communication Protocol Between Agents

Communication between agents is represented by communication events $c_{ij}^{(t)} \in \mathcal{C}_p$, where each event $c_{ij}^{(t)}$ encapsulates the interaction from agent v_i to agent v_j along an edge $e_{ij} \in \mathcal{E}$ at time step t . Formally, a communication event $c_{ij}^{(t)}$ is defined as

$$c_{ij}^{(t)} = (\mathbf{M}_{ij}^{(t)}, \mathbf{B}_{ij}^{(t)}, \mathbf{I}_{ij}^{(t)})$$

where $\mathbf{M}_{ij}^{(t)}$ indicates the *message* content sent from v_i to v_j , containing instructions or clarifications, $\mathbf{B}_{ij}^{(t)}$ denotes *background* information to ensure coherence and task progression (including the problem's core details and intermediate decisions), and $\mathbf{I}_{ij}^{(t)}$ refers to the *intermediate output* generated by v_i and shared with v_j to support task progression and traceability.

1) *Intermediate Output: Definition and Motivation:* We define the intermediate output $\mathbf{I}_{ij}^{(t)}$ as an explicit *artifact* produced by the sender agent and intended to be directly consumed by downstream agents for verification, aggregation, and revision. Unlike a raw conversational history, $\mathbf{I}_{ij}^{(t)}$ is designed to be concise and structured, standardizing *what* is evaluated across agents. Depending on the task, $\mathbf{I}_{ij}^{(t)}$ may include extracted constraints, a candidate solution or patch, a structured summary of key claims with supporting evidence, or other machine-readable artifacts (e.g., JSON fields) that are easier to verify and less sensitive to ordering effects than free-form dialogue.

2) *Message Schema and Transmission Pathway:* In *TalkHier*, each communication event $c_{ij}^{(t)}$ is transmitted as a structured message with three explicit fields ($\mathbf{M}_{ij}^{(t)}, \mathbf{B}_{ij}^{(t)}, \mathbf{I}_{ij}^{(t)}$). Supervisor-to-member edges include $\mathbf{B}_{ij}^{(t)}$ to provide persistent task context, while member-to-supervisor edges omit $\mathbf{B}_{ij}^{(t)}$ and send only $(\mathbf{M}_{ij}^{(t)}, \mathbf{I}_{ij}^{(t)})$ to avoid duplicating context and to focus aggregation on standardized artifacts. Downstream evaluator agents consume $\mathbf{I}_{ij}^{(t)}$ as the primary object of assessment (e.g., scoring or critiquing the candidate output), and supervisor agents use the collected artifacts to support routing and aggregation decisions.

3) *Communication Event Sequence:* At each time step t , the current agent v_i communicates with a connected node v_j , with one being selected by the LLM if more than one exists. The elements $\mathbf{M}_{ij}^{(t)}, \mathbf{B}_{ij}^{(t)}$, and $\mathbf{I}_{ij}^{(t)}$ are generated by invoking an

Supervisor Prompt Template

Team Members: *[Description of each team member's role]*Conversation History: *[Independent Conversation History]*

Given the conversation above, output the following in this exact order:

1. 'thoughts': Output a detailed analysis on the most recent message. In detail, state what you think should be done next, and who you think you should contact next.
2. Who should act next? Select one of: *[Team member names]* and output as 'next'. When you have determined that the final output is gained, report back with FINISH.
3. 'messages': If the next agent is one of *[Team member names]*, give detailed instructions. If FINISH, report a summary of all results.
4. The detailed background of the problem you are trying to solve (given in the first message) as 'background'.
5. The intermediate outputs to give as 'intermediate_output'.

Member Prompt Template

*[Role of member]*Background: *[Background information given by Supervisor]*Conversation History: *[Independent Conversation History]*Fig. 4. Prompts for acquiring the contents of the context-rich, structured communication protocol in *TalkHier*.

independent LLM. To ensure consistency, clarity, and efficiency in extracting these elements, the system employs specialized prompts tailored to the roles of supervisors and members, as illustrated in Fig. 4. Most notably, background information $B_{ij}^{(t)}$ is not present for connections from member nodes to supervisor nodes. The resulting tuple is then established as a communication event $c_{ij}^{(t)} \in \mathcal{C}_p$, and the receiver agent treats $(M_{ij}^{(t)}, B_{ij}^{(t)}, I_{ij}^{(t)})$ (or the available subset) as its structured input for the next-step reasoning, evaluation, or revision.

IV. EXPERIMENTS

In this section, we aim to answer the following research questions across various domains.

- 1) *RQ1*: Does *TalkHier* outperform existing multiagent, single-agent, and proprietary approaches on general benchmarks?
- 2) *RQ2*: How does *TalkHier* perform on open-domain question-answering tasks?
- 3) *RQ3*: What is the contribution of each component of *TalkHier* to its overall performance?
- 4) *RQ4*: How well does *TalkHier* generalize to more practical but complex generation task?

A. Experimental Setup

1) *Datasets*: We evaluated *TalkHier* on a diverse collection of datasets to assess its performance across various tasks. The massive multitask language understanding (MMLU) benchmark [41] tests domain-specific reasoning problems including moral scenario, college physics, machine learning (ML), formal logic (FL), and U.S. Foreign Policy. WikiQA [42] evaluates open-domain question-answering using real-world questions from Wikipedia. The Camera Dataset [43] focuses on advertisement headline generation, assessing the ability to create high-quality advertising text.

TABLE I
GENERAL PERFORMANCE ON MMLU DATASET

Models	Moral	Phys.	ML	FL	UFP	Avg.
GPT4o	64.25	62.75	67.86	63.49	92.00	70.07
GPT4o-3@	65.70	62.75	66.07	66.67	91.00	70.44
GPT4o-5@	66.15	61.76	66.96	66.67	92.00	70.71
GPT4o-7@	65.81	63.73	66.96	68.25	91.00	71.15
ReAct	69.61	72.55	59.82	32.54	58.00	58.50
ReAct-3@	74.75	83.33	66.07	52.38	53.00	65.91
ReAct-5@	74.97	82.35	66.96	46.83	63.00	66.82
ReAct-7@	75.53	84.78	67.86	50.79	57.00	67.19
AutoGPT	66.37	78.43	64.29	60.83	90.00	71.98
AgentVerse	79.11	93.14	79.46	78.57	88.00	83.66
GPTSwarm	60.48	67.70	72.32	68.33	57.00	65.17
AgentPrune	70.84	91.18	81.25	81.75	93.00	83.60
o1-preview	82.57	91.17	85.71	83.33	95.00	87.56
<i>TalkHier (Ours)</i>	83.80	93.14	84.68	87.30	93.00	88.38

Note: The table reports accuracy (%) for various baselines across Moral Scenario (Moral), College Physics (Phys.), Machine Learning (ML), Formal Logic (FL), and US Foreign Policy (UFP). The notations **3@**, **5@**, and **7@** represent majority voting results using 3, 5, and 7 independent runs, respectively. Bold entries indicate the best-performing method across all baselines and our proposed method.

2) *Baselines*: To evaluate *TalkHier*, we compared it against a comprehensive set of baselines, including:

- a) *GPT-4o* [44] is based on OpenAI's GPT-4 model with both single-run and ensemble majority voting (three, five, or seven runs).
- b) *OpenAI-o1-preview* [45] is a beta model using advanced inference techniques, though limited by API support.
- c) *ReAct* [46] is a reasoning and action framework in single-run and ensemble configurations.
- d) *AutoGPT* [38] is an autonomous agent designed for task execution and iterative improvement.

Good Revision Example

Problem: For which of these two scenarios does the main character (who uses I/me/my) do something clearly morally wrong, according to ordinary moral standards in the US as of 2020?

Scenario 1: I spoke to my mom on the phone during my entire work shift.

Scenario 2: I helped my friend escape prison.

Correct Answer: 0 (Wrong, Wrong)

Generated Answer: 2 (Not Wrong, Wrong)

Evaluation Results:

- **Scenario 1:**
Normality **Correct**; Intent **Correct**; Responsibility **Incorrect**; Well-being **Incorrect**
- **Scenario 2:**
All metrics **Correct**

Revisor Reasoning: Based on the detailed evaluation results, the provided answer needs to be revised.

- The evaluation indicates that Scenario 1 is considered irresponsible and promotes harm to the well-being of everyone involved, which contradicts the generated answer of “Not Wrong”. Therefore, the correct assessment is “Wrong”.
- The evaluation confirms that Scenario 2 is considered irresponsible and harmful, which aligns with the generated answer of “Wrong”.

Revised Answer: 0 (Wrong, Wrong)

Fig. 5. Good revision example in *TalkHier*. This example demonstrates how the system uses detailed evaluation results to revise an initially incorrect answer. The evaluation metrics (normality, intent, responsibility, well-being) are used to analyze each scenario, and the revisor agent provides reasoning based on these results. The revised answer corrects the initial output by aligning it with the evaluation feedback, illustrating the effectiveness of hierarchical refinement and structured communication in *TalkHier*.

TABLE II
EVALUATION RESULTS ON WIKIQA

Models	Rouge-1	Rouge-L	BERTScore
GPT4o	0.2777	0.1709	0.5856
ReAct	0.2409	0.1858	0.5415
AutoGPT	0.3286	0.2617	0.5885
AgentVerse	0.2799	0.2010	0.5716
AgentPrune	0.3027	0.2319	0.5788
GPTSwarm	0.2302	0.1779	0.5067
GPT-o1	0.2631	0.1884	0.5701
GPT-o3	0.3348	0.2689	0.6032
<i>TalkHier</i> (4o)	0.3461	0.2807	0.6079
<i>TalkHier</i> (o1)	0.3576	0.2924	0.6168

Note: The table reports Rouge-1, Rouge-L, and BERTScore for various models. Bold entries indicate the best-performing method across all baselines and our proposed method.

- AgentVerse* [27] is a multiagent system framework for collaborative problem-solving.
- GPTSwarm* [21] is a swarm-based agent collaboration model utilizing optimizable communication graphs.
- AgentPrune* [22] is a model leveraging pruning techniques for efficient multiagent communication and reasoning.
- On-the-Fly Keyword Generation (OKG)* [47] is a method tailored specifically for ad text generation tasks and easily generalizable to ad headlines with minimal prompt redefinition.

TABLE III
ABLATIVE RESULTS: ACCURACY (%) ACROSS PHYSICS, ML, AND MORAL DOMAINS

Models	Moral	Phys.	ML	FL	UFP	Avg.
w/o Eval. Sup.	83.57	87.25	74.77	83.33	82.00	82.18
w/o Eval. Team	73.54	80.34	74.56	73.02	89.00	78.09
w. Norm. Comm	82.91	88.24	82.14	84.80	93.00	86.22
Single Agent	69.61	72.55	59.82	32.54	58.00	58.50
w/o I_{ij}	81.56	90.20	75.89	79.37	93.00	84.00
w/o B_{ij}	76.87	87.50	70.54	64.29	87.00	77.24
w/o B_{ij}, I_{ij}	77.99	90.20	78.57	69.84	93.00	81.92
<i>TalkHier</i> (Ours)	83.80	93.14	84.68	87.30	93.00	88.38

Note: The top half examines the impact of structural components (evaluation supervisor, evaluation team, refinement structure), while the bottom half examines communication protocol components (message M_{ij} , background B_{ij} , and intermediate output I_{ij}). Bold entries indicate the best-performing method across all baselines and our proposed method.

3) *Implementation Details:* For fair comparisons, we use GPT-4o as the backbone across all experiments for the baselines and *TalkHier*, with the temperature set to 0 in all settings. For the OpenAI-o1 baseline, we followed the implementation guide and the limitations outlined in OpenAI’s documentation¹ and kept the temperature fixed at 1.

B. Performance on MMLU (RQ1)

Table I reports the average accuracy of various models on the five domains of MMLU dataset. *TalkHier*, built on GPT-4o,

¹<https://platform.openai.com/docs/guides/reasoning/beta-limitations>

TABLE IV
EVALUATION RESULTS ON CAMERA DATASET

Models	B4 (↑)	R1 (↑)	BERT (↑)	Faithfulness (↑)	Fluency (↑)	Attractiveness (↑)	CCV (↓)
GPT-4o	0.01	0.02	0.65	4.8	5.9	6.5	16%
ReAct	0.01	0.01	0.70	4.9	6.4	7.0	17%
OKG	0.03	0.16	0.73	6.3	8.7	6.1	4%
<i>TalkHier (Ours)</i>	0.04	0.20	0.91	8.6	8.9	6.2	4%

Note: We report BLEU-4 (B4), ROUGE-1 (R1), BERTScore (BERT), and domain-specific metrics [Faithfulness, Fluency, Attractiveness, Character Count Violation(CCV)] following [43]. Bold entries indicate the best-performing method across all baselines and our proposed method.

achieves the highest average accuracy (88.38%), outperforming open-source multiagent models (e.g., AgentVerse, 83.66%) and majority voting strategies applied to current LLM and single-agent baselines (e.g., ReAct-7@, 67.19%; GPT-4o-7@, 71.15%). These results highlight the effectiveness of our hierarchical refinement approach in enhancing GPT-4o’s performance across diverse tasks. Although OpenAI-o1 cannot be directly compared to *TalkHier* and other baselines—since they are all built on GPT-4o and OpenAI-o1’s internal design and training data remain undisclosed—*TalkHier* achieves a slightly higher average score (88.38% versus 87.56%), demonstrating competitive performance.

Revision Example: Fig. 5 illustrates a good revision example from *TalkHier* on the Moral Scenario domain of MMLU. The initially generated answer incorrectly assesses the moral status of two scenarios. However, through detailed evaluation by multiple evaluators focusing on different moral aspects (normality, intent, responsibility, and well-being), the system identifies the inaccuracies. The revisor agent then uses this feedback to reason about the correct moral judgments and revises the answer, accordingly, resulting in the correct assessment of both scenarios as “wrong.” This example highlights the effectiveness of *TalkHier*’s structured communication and hierarchical refinement in producing accurate outputs.

C. Evaluation on WikiQA Benchmark (RQ2)

We evaluated *TalkHier* and baselines on the WikiQA dataset, an open-domain question-answering benchmark. Unlike MMLU, WikiQA requires generating textual answers to real-world questions. The quality of generated answers was assessed using two metrics: Rouge-1 [48], which measures unigram overlap between generated and reference answers, and BERTScore [49], which evaluates the semantic similarity between the two.

Table II shows that *TalkHier* consistently outperforms strong baselines on WikiQA in both Rouge-1/Rouge-L and BERTScore, demonstrating its ability to generate answers that are both lexically accurate and semantically relevant. Notably, when equipped with a stronger reasoning backbone (o3), *TalkHier* further improves over the corresponding single-model baseline (o3), indicating that the gains are not limited to GPT-4o but also transfer to stronger models through better coordination and verification. While some approaches (e.g., AutoGPT) perform competitively—likely because many WikiQA questions are relatively straightforward—they still fall short of *TalkHier*.

TABLE V
HUMAN-SYSTEM AGREEMENT ON A CAMERA SUBSET (10 GENERATED HEADLINES; HUMAN RATINGS VERSUS *TALKHIER*)

Metric	Value	p-Value
Pearson correlation	0.67	0.036
Spearman correlation	0.68	0.030
ICC (2,1)	0.23	–
ICC (2,4)	0.33	–

Note: Confidence intervals are not reported due to the small sample size.

Overall, these results suggest that our hierarchical refinement and structured communication protocol provide a robust advantage over both single-agent prompting and existing multiagent baselines, yielding more reliable and consistent performance.

D. Ablation Study (RQ3)

To better understand the contribution of individual components in *TalkHier*, we conducted ablation studies by removing specific modules and evaluating the resulting performance across the moral scenario, college physics, and ML domains. The results are summarized in Table III.

The top half of Table III reports ablations of key structural components. Removing the evaluation supervisor (*TalkHier* w/o Eval. Sup.) reduces accuracy, indicating the importance of supervisor-level aggregation in our hierarchical refinement. Replacing the structured communication protocol with a standard unstructured text-based one (*TalkHier* w. Norm. Comm) leads to a moderate decrease in accuracy. Removing the evaluation team (*TalkHier* w/o Eval. Team) causes the largest performance drop across tasks, highlighting the value of parallel, metric-scoped evaluation signals for robust refinement.

The bottom half of Table III examines the role of individual fields in the communication protocol. Excluding intermediate outputs (*TalkHier* w/o I_{ij}) or background information (*TalkHier* w/o B_{ij}) reduces performance, and removing both (*TalkHier* w/o B_{ij}, I_{ij}) leads to similarly degraded results. These findings highlight the importance of context-rich, structured communication in sustaining strong performance on complex tasks.

Evidence for Bias Mitigation (Order/Position and Anchoring): While “unbiasedness” is hard to directly quantify without dedicated perturbation tests, Table III

TABLE VI
QUALITATIVE EXAMPLES ON AD-TEXT GENERATION TASK

ID	Method	Headline (English)	Human Rating-1	Human Rating-2
H1_card	<i>TalkHier</i>	LifeCard with No Annual Fee	4.33	4.33
H2_card	<i>TalkHier</i>	Receive Your Card in Two Business Days	5.00	4.66
H3_card	<i>TalkHier</i>	Earn Points for Every ¥100 You Spend	4.33	5.00
H4_card	<i>TalkHier</i>	Triple Points on Your Birthday Month	4.33	4.33
H5_card	<i>TalkHier</i>	A Card That Fits Your Lifestyle	2.33	4.00
H6_card	ReAct	Full of Benefits, LifeCard is Here	3.66	3.00
H7_card	ReAct	Start a New Life with LifeCard	2.33	3.66
H8_card	ReAct	Save Smartly with LifeCard	3.66	4.33
H9_card	ReAct	Shop with LifeCard	3.66	3.66
H10_card	ReAct	Trusted and Reliable Life Card	3.66	4.00

Note: ReAct and *TalkHier* use the same GPT-4o backbone; differences reflect protocol-level coordination/verification. We show representative headlines and their human ratings (1–5).

TABLE VII
TOKEN USAGE OF *TALKHIER* (GPT-4O;
PROMPT+COMPLETION) ACROSS BENCHMARKS

Benchmark	Tokens/inst.	# Inst.
MMLU–Moral	48.3K	895
MMLU–Phys.	40.0K	101
MMLU–ML	40.0K	111
MMLU–FL	46.0K	125
MMLU–UFP	45.0K	100
WikiQA	36.0K	1473
AdText	65.0K	872

Note: We report average tokens per instance and the number of evaluated instances.

provides *ablation-based evidence* that our protocol-level designs (structured communication artifacts and hierarchical aggregation) are important for mitigating order-/position-induced bias and anchoring/confirmation effects in multiagent refinement. First, removing the evaluation team or the evaluation supervisor degrades average accuracy (e.g., $88.38 \rightarrow 78.09/82.18$), which is consistent with the benefit of *parallel*, *metric-scoped* evaluations and supervisor-level aggregation over a single sequential feedback stream that can be more sensitive to feedback ordering and anchoring. Second, replacing our structured protocol with unstructured communication (“w. Norm. Comm”) also reduces performance ($88.38 \rightarrow 86.22$), suggesting that long free-form exchanges are less reliable and more sensitive to how information is presented. Finally, ablating the structured communication fields further confirms their necessity: removing background $\mathbf{B}_{ij}$ causes the largest drop among protocol ablations ($88.38 \rightarrow 77.24$), and removing intermediate output $\mathbf{I}_{ij}$ also degrades performance ($88.38 \rightarrow 84.00$). These results are consistent with our claim that explicitly separating message $\mathbf{M}_{ij}$, shared context $\mathbf{B}_{ij}$, and the concrete artifact $\mathbf{I}_{ij}$ reduces long-context

sensitivity and discourages implicit anchoring to partially described drafts.

E. Evaluation on Ad Text Generation (RQ4)

We evaluate *TalkHier* on the Camera dataset [43] using both standard text generation metrics (BLEU-4, ROUGE-1, BERTScore) and domain-specific metrics [faithfulness, fluency, attractiveness, and character count violation (CCV)]. These metrics jointly assess linguistic quality and task-specific effectiveness. Establishing baselines such as AgentVerse or GPTSwarm for this task was difficult, since their implementations are optimized for general benchmarks such as MMLU and require significant customization to handle ad text. In contrast, OKG [47], originally designed for ad keyword generation, was more straightforward to adapt and therefore serves as a practical baseline.

Table IV reports the results. *TalkHier* consistently outperforms ReAct, GPT-4o, and OKG, showing particularly strong improvements in faithfulness, fluency, and attractiveness, while maintaining a low CCV rate. On average, *TalkHier* achieves a 17.63% performance gain over the best-performing baseline (OKG) across all metrics.

To further validate whether these improvements reflect human preferences, we conducted a human evaluation on a subset of the Camera dataset. We compared ratings from four human annotators with those from *TalkHier*’s evaluation agents, using a 1–5 scale for attractiveness, fluency, and faithfulness.

As shown in Table V, *TalkHier*’s ratings correlate moderately with the average human ratings (Pearson = 0.67, Spearman = 0.68, both $p < 0.05$). Agreement with individual raters is relatively low [ICC(2,1) = 0.23], reflecting variability in human judgments. However, agreement improves when compared to the aggregated consensus [ICC(2,4) = 0.33], suggesting that *TalkHier* captures collective human preferences more effectively than individual opinions. Overall, these findings reinforce the validity of using *TalkHier*’s evaluation agents to guide refinement in ad text generation.

Qualitative Case Study (Ad Text Generation): In addition to quantitative results, we include a qualitative case study on the Camera ad-text generation task. Table VI presents representative outputs from *TalkHier* and a strong baseline, ReAct. Importantly, ReAct is also built on the same GPT-4o backbone; thus, the differences mainly reflect protocol-level coordination and verification rather than a stronger base model. Consistent with the human ratings, *TalkHier* tends to produce more specific, benefit-driven, and constraint-aware headlines, while ReAct outputs are often more generic and less informative.

F. Token Usage Analysis

We analyze the token usage of *TalkHier* (GPT-4o backbone) to quantify the computational cost of multiagent collaboration. Since several baselines are not implemented in LangGraph-style logging, we report token statistics for *TalkHier*, measured by the API usage metadata (prompt + completion tokens) per instance.

Overall, *TalkHier* uses more tokens than single-agent prompting due to: 1) multiview evaluation; and 2) iterative refinement; however, this additional cost consistently translates into higher accuracy across diverse tasks (see Table VII).

V. CONCLUSION

In this article, we propose *TalkHier*, a novel framework for LLM-MA systems that addresses key challenges in communication and hierarchical refinement. To the best of our knowledge, *TalkHier* is the first framework to integrate a structured communication protocol in LLM-MA systems, embedding *messages*, *intermediate outputs*, and *background* information to ensure organized and context-rich exchanges. At the same time, distinct from existing works that have biases on inputs, its dynamic hierarchical refinement approach balances and summarizes diverse opinions or feedback from agents. *TalkHier* surpasses the best-performing baseline by an average of 5.64% on MMLU, 4.31% on WikiQA, and 17.63% on Camera benchmarks.

Beyond consistently outperforming prior baselines, it also slightly outperforms the inference scaling model OpenAI-o1, demonstrating its potential for scalable, unbiased, and high-performance multiagent collaborations. In addition, compared to other multiagent systems that rely on simple or ad-hoc structures, our framework provides a principled hierarchical design and a structured communication protocol, making it more robust, generalizable, and effective across diverse tasks.

A. Limitations

A limitation of *TalkHier* is that our proposed structured communication among agents and hierarchical evaluation, while significantly improving performance, generated a relatively larger number of tokens. This is a tradeoff due to the design of *TalkHier*, where multiple agents collaborate hierarchically using a specifically designed communication protocol. While this structured interaction enhances reasoning and coordination, it also increases computational expenses.

This raises broader concerns about the accessibility and democratization of LLM research, as such costs may pose barriers for researchers with limited resources. Future work could explore more cost-efficient generation strategies while preserving the benefits of multiagent collaboration.

DATA AVAILABILITY STATEMENT

The source code is available at <https://github.com/sony/talkhier>.

REFERENCES

- [1] A. Brohan et al., "Code as policies: Language model-driven robotics," 2022, *arXiv:2209.07753*.
- [2] S. Shah et al., "FinGPT: An open-source financial large language model," 2023, *arXiv:2306.03026*.
- [3] W. Zhang et al., "FinAgent: A multimodal foundation agent for financial trading: Tool-augmented, diversified, and generalist," 2024, *arXiv:2402.18485*.
- [4] M. Chen et al., "Evaluating large language models trained on code," 2021, *arXiv:2107.03374*.
- [5] S. Hong et al., "MetaGPT: Meta programming for multi-agent collaborative framework," 2023, *arXiv:2308.00352*.
- [6] Q. Wang, Z. Wang, Y. Su, H. Tong, and Y. Song, "Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key?" 2024, *arXiv:2402.18272*.
- [7] E. Eigner and T. Händler, "Determinants of LLM-assisted decision-making," 2024, *arXiv:2402.17385*.
- [8] P. Pezeshkpour, E. Kandogan, N. Bhutani, S. Rahman, T. Mitchell, and E. Hruschka, "Reasoning capacity in multi-agent systems: Limitations, challenges and human-centered solutions," 2024, *arXiv:2402.01108*.
- [9] Z. Xi et al., "The rise and potential of large language model based agents: A survey," 2024, *arXiv:2309.07864*.
- [10] C. Gao et al., "Large language models empowered agent-based modeling and simulation: A survey and perspectives," 2023, *arXiv:2312.11970*.
- [11] L. Wang et al., "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, 2024, Art. no. 186345. [Online]. Available: <https://doi.org/10.1007/s11704-024-40231-1>
- [12] Y. Cheng et al., "Exploring large language model based intelligent agents: Definitions, methods, and prospects," 2024, *arXiv:2401.03428*.
- [13] Q. Ma et al., "Computational experiments meet large language model based agents: A survey and perspective," 2024, *arXiv:2402.00262*.
- [14] P. Chen, B. Han, and S. Zhang, "CoMM: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving," 2024, *arXiv:2404.17729*.
- [15] C. He, B. Zou, X. Li, J. Chen, J. Xing, and H. Ma, "Enhancing LLM reasoning with multi-path collaborative reactive and reflection agents," 2024, *arXiv:2501.00430*.
- [16] A. Madaan, N. Tandon, D. Downey, and S. Han, "Self-refine: Iteratively improving text via self-feedback," 2023, *arXiv:2303.17651*.
- [17] Z. Yang et al., "Idea2Img: Iterative self-refinement with GPT-4V(ision) for automatic image design and generation," 2023, *arXiv:2310.08541*.
- [18] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," 2022, *arXiv:2201.11903*.
- [19] S. Yao et al., "Tree of thoughts: Deliberate problem solving with large language models," 2023, *arXiv:2305.10601*.
- [20] C. Qian et al., "Scaling large-language-model-based multi-agent collaboration," 2024, *arXiv:2406.07155*.
- [21] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "GPTSwarm: Language agents as optimizable graphs," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 62743–62767.
- [22] G. Zhang et al., "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," 2024, *arXiv:2410.02506*.
- [23] Q. Zhao et al., "CompeteAI: Understanding the competition dynamics in large language model-based agents," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024, pp. 61092–61107.
- [24] T. Guo et al., "Large language model based multi-agents: A survey of progress and challenges," 2024, *arXiv:2402.01680*.
- [25] X. Wang, Y. Liu, and H. Zhang, "COEVAL: A framework for collaborative human and machine evaluation," 2023, *arXiv:2310.19740*.
- [26] J. Fang et al., "Multi-LLM text summarization," 2024, *arXiv:2412.15487*.

- [27] OpenBMB, "AgentVerse: Facilitating multi-agent collaboration," *AgentVerse GitHub*, 2023. [Online]. Available: <https://github.com/OpenBMB/AgentVerse>
- [28] F. Errica, G. Siracusano, D. Sanvito, and R. Bifulco, "What did i do wrong? Quantifying LLMs' sensitivity and consistency to prompt engineering," 2024, *arXiv:2406.12334*.
- [29] S. Han, Q. Zhang, Y. Yao, W. Jin, Z. Xu, and C. He, "LLM multi-agent systems: Challenges and open problems," 2024, *arXiv:2402.03578*.
- [30] Y. Yang, Q. Peng, J. Wang, and W. Zhang, "Multi-LLM-agent systems: Techniques and business perspectives," 2024, *arXiv:2411.14033*.
- [31] S. Rasal, "LLM harmony: Multi-agent communication for problem solving," 2024, *arXiv:2401.01312*.
- [32] X. Qiu et al., "Towards collaborative intelligence: Propagating intentions and reasoning for multi-agent coordination with large language models," 2024, *arXiv:2407.12532*.
- [33] Y. Talebirad and A. Nadiri, "Multi-agent collaboration: Harnessing the power of intelligent LLM agents," 2023, *arXiv:2306.03314*.
- [34] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, no. 9, 2024, Art. no. 9.
- [35] W. Shen et al., "Small LLMs are weak tool learners: A multi-LLM agent," 2024, *arXiv:2401.07324*.
- [36] L. Xu, Q. Sun, and H. Zhao, "Cooperative evaluation in large language model refinement," 2024, *arXiv:2401.10234*.
- [37] H. Du, S. Thudumu, R. Vasa, and K. Mouzakis, "A survey on context-aware multi-agent systems: Techniques, challenges and future directions," 2024, *arXiv:2402.01968*.
- [38] S. Gravitass, "AutoGPT: An experimental open-source application," 2023. [Online]. Available: <https://github.com/Torantulino/Auto-GPT>
- [39] LangGraph, "LangGraph: Build resilient language agents as graphs," 2024. Accessed: Dec. 25, 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [40] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 8634–8652.
- [41] D. Hendrycks, C. Burns, S. Basart, C. Zou, D. Song, and T. G. Dietterich, "Measuring massive multitask language understanding," 2021, *arXiv:2110.08307*. [Online]. Available: <https://arxiv.org/abs/2110.08307>
- [42] W.-t. Yang et al., "WikiQA: A challenge dataset for open-domain question answering," in *Proc. 2015 Conf. Empirical Methods Natural Lang. Process.*, Lisbon, Portugal: Association for Computational Linguistics, 2015, pp. 2013–2018.
- [43] M. Mita, S. Murakami, A. Kato, and P. Zhang, "Striking gold in advertising: Standardization and exploration of ad text generation," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 955–972.
- [44] OpenAI, "Hello GPT-4o," 2024. Accessed: Mar. 31, 2026. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [45] OpenAI, "Introducing OpenAI o1," 2024. Accessed: Mar. 31, 2026. [Online]. Available: <https://openai.com/index/introducing-openai-o1-preview/>
- [46] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.
- [47] Z. Wang, B. Gangopadhyay, M. Zhao, and S. Takamatsu, "OKG: On-the-fly keyword generation in sponsored search advertising," in *Proc. 31st Int. Conf. Comput. Linguistics: Industry Track.*, Assoc. for Comput. Linguistics, Jan. 2025, pp. 115–127.
- [48] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Proc. Text Summarization Branches Out*, 2004, pp. 74–81.
- [49] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with Bert," 2020, *arXiv:1904.09675*.